F21RO (Intelligent Robotics)

NEURAL NETWORK INVERSE KINEMATICS

Group 26

MSc Artificial Intelligence
Heriot-Watt University

2026–2027

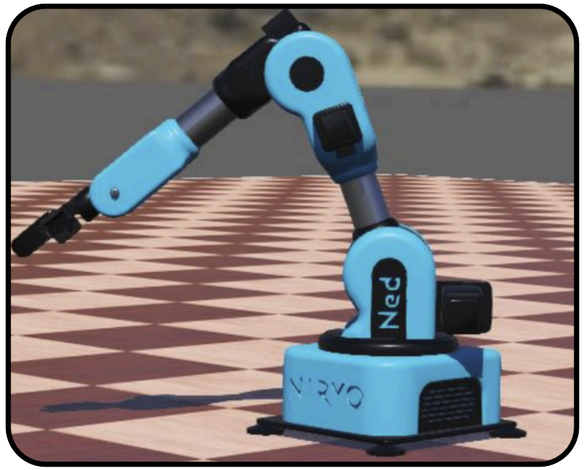
Team Members

Group 26

- Yash Nadkarni
- Gunasai Anumulapally
- Shyam Agarwal
- Manav Parmar

Introduction & Aim

- Artificial Neural Network (ANN) to approximate inverse kinematics (IK) for robotic manipulators.
- Implement a single shared neural network capable of controlling two different robot arms: Niryo Ned & Epson SCARA.
- Predict the required joint values from a desired Tool Centre Point (TCP) position (x, y, z).
- Train the network using datasets generated through analytical Forward Kinematics (FK).
- Investigate the influence of network architecture, hyperparameters and dataset size on prediction accuracy.
- Evaluate the network using analytical FK and the Webots simulator.



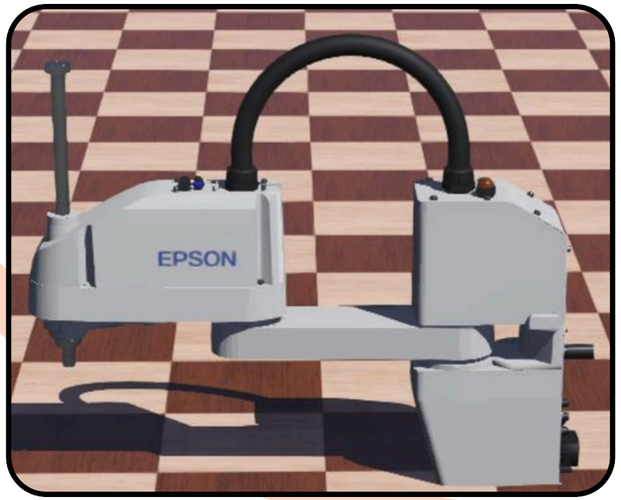


Image Reference: Coursework Specification

Pipeline

1. **Foward Kinematics**

2. **Dataset Generation**
   - Defining Limits; Elbow-Up Only
   - 80/20 Training & Validation Split
   - Nested Datasets
   - 500 Test Sets
   - Normalized Target Joint Values

3. **Preprocessing**
   - is_ned, is_scara flags
   - Derived Radial Distance as 6th Input

4. **Neural Network**
   - Hyper-parameter Tuning
   - Clipping Illegal Joints

5. **Evaluation**
   - TCP Position Error using FK & Webots
   - Joint Errors (Angular & Squared Displacement)
   - Illegal Joint Predictions

Forward Kinematics

- **Ned:** Three revolute joints (q1, q2, q3); wrist joints fixed.
- **SCARA:** Two revolute joints (q1, q2) and one prismatic joint (q4).
- Calculated TCP position (x, y, z) from known joint configurations.
- Validated analytical FK against Webots TCP supervisor measurements.
- Used the validated FK models to generate training data.

```
#combine the fixed shoulder to elbow offset
#with the elbow to TCP vector rotated by q3
wx = A + U * math.cos(q3) - V * math.sin(q3)
wy = B + U * math.sin(q3) + V * math.cos(q3)

#apply the shoulder rotation q2
#r is the horizontal distance from the q1 axis
r = wy * math.cos(q2) + wx * math.sin(q2)
z = H - wy * math.sin(q2) + wx * math.cos(q2)

#apply the waist rotation q1
#ned has q1 = 0 pointing along +Y
x = -r * math.sin(q1)
y = r * math.cos(q1)
```

TCP from Ned FK

$$\mathbf{p} = \begin{bmatrix} 0.060 \\ 0 \\ 0.220 \end{bmatrix} + R_z(q_1) \left(\begin{bmatrix} 0.310 \\ 0.005 \\ -0.004 \end{bmatrix} + R_z(q_2) \begin{bmatrix} 0.2545 \\ -0.005 \\ q_4 - 0.034 \end{bmatrix} \right)$$

TCP from Scara FK

Image Reference: Coursework Specification

Dataset Generation

- Sampling valid joint configurations within predefined limits.
- Elbow-Up Configuratioin
  - q3 >= 0 for Ned
  - q2 >= 0 for SCARA
- SCARA prismatic joint ground collision
  - Fix: q4 limit changed from -0.2 m to -0.17 m
- Created three dataset sizes per robot to investigate the effect of training data volume.
- Datasets are nested.
  - **Small ⊂ Medium ⊂ Large**
- 80/20 training-validation split per dataset.
- Independent test set of 500 samples per robot.
- Combined Ned and SCARA datasets by size for training one shared network.

Dataset	Size	Training:Validation (80:20 Split)
Small	1000	800 : 200
Medium	5000	4000 : 1000
Large	10000	8000 : 2000

Dataset Size & Split per Robot

Dataset	Size
Small	2000
Medium	10000
Large	20000

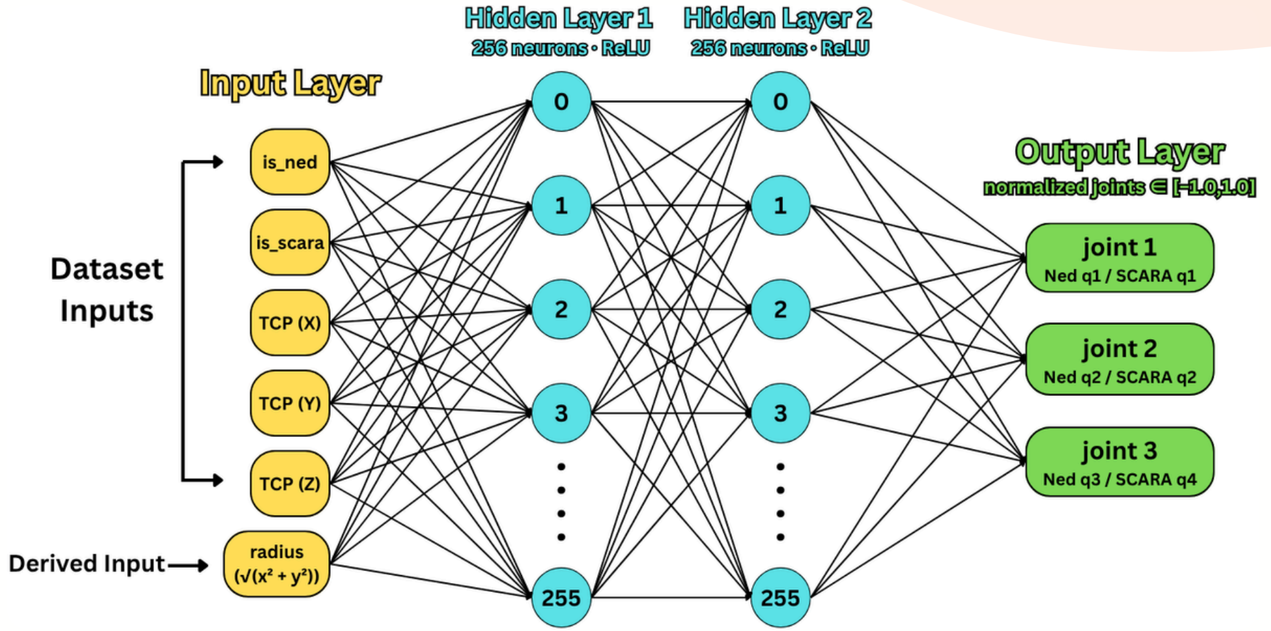
Combined Dataset Sizes

Pre-Processing

- Added binary robot identifier flags: is_ned and is_scara.
- Normalized joint values to [-1, 1] using robot-specific joint limits.
- Retained raw TCP coordinates (x, y, z) in metres.
- Added Horizontal Radial Distance $\sqrt{}(x^2 + y^2)$, derived from x & y inputs as an additional input feature to ease learning.
- Shuffled the combined dataset to reduce ordering bias.

Neural Network

- Problem Type: **Regression** → Scikit-learn's MLPRegressor
- Architecture: 6 input neurons → 256 → 256 hidden neurons → 3 output neurons.
- Inputs: Robot identifier flags (is_ned, is_scara), TCP coordinates (x, y, z), and Horizontal Radial Distance $\sqrt{(x^2 + y^2)}$
- Outputs: Three normalized joint predictions
- Hyperparameters: ReLU activation, Adam optimizer, learning rate of 0.001.
- Post-processing: **Clip predictions** to [-1, 1] and denormalize using joint limits.
- Conducted **15+** Hyperparameter Tuning experiments.
- **Final Design Choice:** 256–256 architecture to balance accuracy across both robots without excessive complexity.



Neural Network Architecture

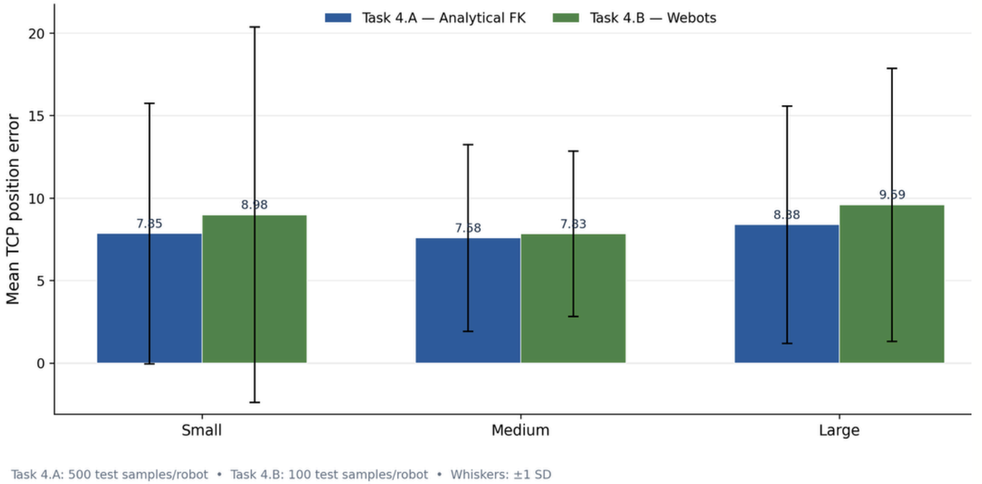
Layers	Activation	Learning Rate	Optimizer	Normalized Inputs	Radius Parameter	Clipping
64, 64	ReLU	0.001	Adam	True	False	True
128, 128	ReLU	0.0005	Adam	True	False	True
128, 128	Tanh	0.001	Adam	False	False	True
256, 256	**ReLU**	**0.001**	**Adam**	**False**	**True**	**True**

Subset of Hyperparameter Tuning Experiments

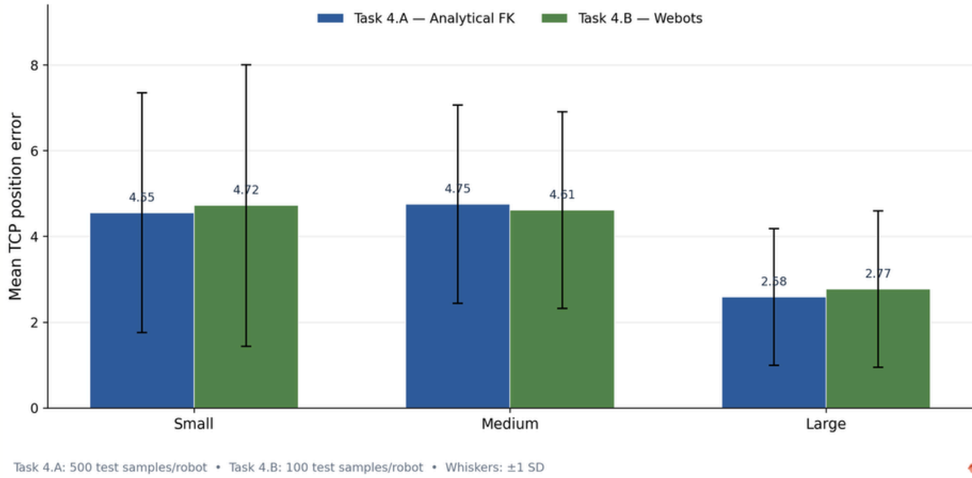
DEMO

Evaluation: TCP Position Accuracy

- Task 4.A — **Analytical Forward Kinematics Evaluation**
  - Predicted joint values evaluated using analytical FK.
  - Compared calculated TCP positions against target coordinates.
  - 500 independent test samples per robot.
- - Task 4.B — **Webots Evaluation**
  - Applied NN predicted joint values to both robots in Webots.
  - Measured actual TCP positions using the TCP Supervisor.
  - 100 test samples per robot.
- Performance Metrics: **Mean** TCP Euclidean error (mm) and **Standard Deviation**.
- **Key Findings**:
  - SCARA consistently achieved lower TCP errors than Ned.
  - Increasing dataset size improved SCARA's accuracy, but not consistently Ned's.
  - Both evaluations produced broadly consistent results.
  - Minor discrepancies from motor settling, simulation tolerances and different test sample sizes.



TCP Position Error for Ned



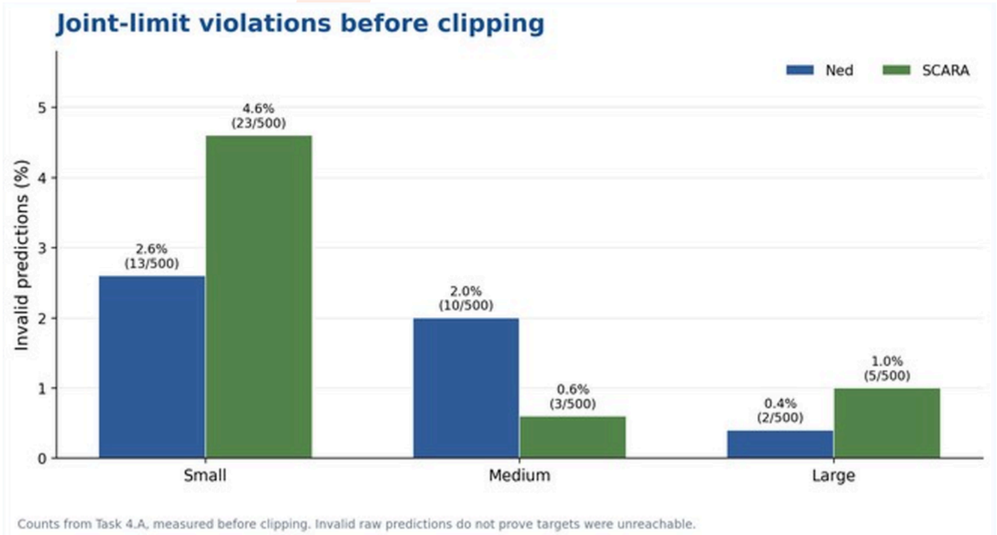
TCP Position Error for SCARA

Evaluation: Joint Accuracy & Limit Violations

- Joint Prediction Error:
  - Revolute joints measured using **angular prediction** errors.
  - SCARA's prismatic joint evaluated using **squared displacement** error (m^2).
- Joint Error Findings:
  - Joint 1: SCARA achieved lower mean error than Ned.
  - Joint 2: Ned achieved lower mean error than SCARA.
  - Third Joint: Both achieved relatively low errors, but direct comparison is inappropriate.
- Joint-Limit Violations:
  - Examined the percentage of raw NN predictions exceeding permitted joint limits before clipping.
  - Both robots exhibited relatively **low violation rates**.
  - Violation percentages decreased as training dataset size increased.
  - **Clipping** ensured predicted commands remained within the specified joint limits.

Robot	Model	Joint 1 (rad) Mean ± Std	Joint 2 (rad) Mean ± Std	Joint 3 Mean ± Std
Ned	Small	−0.02818 ± 0.13890	0.01302 ± 0.01254	0.00536 ± 0.00759 rad
Ned	Medium	−0.01919 ± 0.03844	0.01489 ± 0.01004	0.00360 ± 0.00484 rad
Ned	Large	0.02624 ± 0.04913	0.01074 ± 0.00804	0.00058 ± 0.00509 rad
SCARA	Small	−0.01090 ± 0.01865	0.02646 ± 0.04344	$6.8 \times 10^{-7} \pm 5.5 \times 10^{-6}$ m^2
SCARA	Medium	−0.01804 ± 0.01366	0.02196 ± 0.02738	$1.6 \times 10^{-7} \pm 4.3 \times 10^{-7}$ m^2
SCARA	Large	−0.00826 ± 0.01046	0.00967 ± 0.02232	$1.6 \times 10^{-7} \pm 1.25 \times 10^{-6}$ m^2

Joint Prediction Errors



Joint Limit Violation Rates

Critical Discussion & Conclusion

- **Key Findings**
  - **Dataset size ≠ guaranteed accuracy**: SCARA performed best with the large dataset, while Ned performed best with the medium dataset.
  - Larger and deeper architectures did not consistently improve performance.
  - Introducing **horizontal radial distance** significantly reduced validation error.
  - Shared **model trade-offs**: Improvements for one robot sometimes negatively affected the other.
- **Challenges & Limitations**
  - **IK ambiguity**: Multiple joint configurations can produce similar TCP positions, complicating training.
  - **Joint constraints**: Raw predictions occasionally exceeded joint limits, requiring clipping.
  - **Reachability**: The network cannot explicitly determine whether a target is reachable.
  - Implementation challenges: Coordinate-frame consistency, FK validation and motor settling during evaluation.
- **Conclusion**
  - Successfully developed a shared ANN-based IK model for Ned and SCARA.
  - Demonstrated accurate TCP positioning and target tracking in Webots.
  - A single MLP can approximate IK for different robotic kinematic structures.

Thank you!